POSITIONAL STEGANOGRAPHY

`fog` — a frequency-preserving word wall

`fog` hides a UTF-8 byte string in the **order** of a wall of nearly identical words. The only tokens are the 30 variants

$$\Sigma = \{\texttt{hell} + \texttt{o}^k \mid 1 \le k \le 30\}.$$

For example, `hello`, `helloo`, and `hellooo` are different symbols to the algorithm, but visually form a uniform wall. Embedding changes only the order of tokens: the count of every variant is exactly preserved.

> `fog` is positional steganography, **not encryption**. Anyone who knows or guesses the password can read the embedded payload. Encrypt sensitive data before embedding it.

Properties at a glance

- The wall is a sequence of `hello`-like tokens; its payload is not stored in a token value or a separate metadata field.
- Each usable token pair carries one bit by its relative order.
- The histogram of token variants is identical before and after embedding.
- A password and that invariant histogram reconstruct the same pairs at both ends, without retaining the original carrier.
- The Python encoder is `generator.py`; the browser decoder is `decoder.html` and `decoder.js`.

Algorithm

Let a wall of length N be

$$W = (w_0, w_1, \ldots, w_{N-1}) \in \Sigma^N.$$

1.Create the carrier

The generator samples every position independently and uniformly from the 30 variants:

$$\Pr[w_i = v] = \frac{1}{30}, \qquad v \in \Sigma.$$

`generate_carrier(n)` uses `secrets.SystemRandom()` by default. Supplying `wall_seed` instead makes the carrier reproducible for demonstrations and tests.

2.Derive an order-invariant salt

For each variant v_j, define its count:

$$f_j(W) = \sum_{i=0}^{N-1} \mathbf{1}[w_i = v_j].$$

The generator serializes the 30 counts in the fixed variant order, each as an unsigned 32-bit big-endian integer, and hashes the resulting 120 bytes:

$$S(W) = \mathrm{SHA\text{-}256}(\mathrm{BE}\,32(f_1) \,\|\, \cdots \,\|\, \mathrm{BE}\,32(f_{30})).$$

This value is a salt, not a secret. Its purpose is to bind the password-derived pairing to this particular multiset of words.

The key fact is that swapping positions does not change the count of any token. For every permutation ρ of the indices,

$$f_j(\rho(W)) = f_j(W) \quad \text{and therefore} \quad S(\rho(W)) = S(W).$$

That invariant is why the decoder can rebuild the same salt after the message has been embedded.

3.Use the password to choose pairs

Given password K, the implementation produces a deterministic byte stream:

$$R_c = \text{SHA-256}(S(W) \parallel \text{UTF 8}(K) \parallel \text{BE 64}(c)), \quad c = 0, 1, 2, \ldots$$

It consumes this stream to run Fisher–Yates over `[0, 1, ..., N-1]`. Random integers use rejection sampling, so the selected index is unbiased. The result is a password- and wall-specific permutation

$$\pi_{K,S} = (i_0, i_1, \ldots, i_{N-1}).$$

Consecutive elements form the candidate pairs:

$$P = ((i_0, i_1), (i_2, i_3), \ldots).$$

If N is odd, the final unpaired index is ignored. A pair holding equal token variants is skipped because its order cannot encode a bit.

4.Encode the payload

The byte payload M is converted to bits as

$$B = \text{BE 32}(|M|) \parallel M.$$

The 32-bit prefix records the byte length; it is followed by the bits of the message, most significant bit first in each byte.

For a usable pair (a, b), define the bit represented by its order as

$$\text{bit}(a, b) = \begin{cases} 0 & \text{if } |w_a| > |w_b|, \\ 1 & \text{if } |w_a| < |w_b|. \end{cases}$$

To write the next payload bit, the encoder leaves the pair in place if it already represents that bit; otherwise it swaps `wall[a]` and `wall[b]`. A swap changes no frequency, so

the salt and password-derived pairing remain stable.

For example, let the carrier be represented by variant ranks:

$$W = (7, 8, 9, 3, 2, 4), \qquad P = ((0, 3), (1, 4), (2, 5)).$$

Ignoring the length prefix, encode `101`:

Pair	Values before	Current bit	Target	Action
(0,3)	(7,3)	0	1	swap
(1,4)	(8,2)	0	0	keep
(2,5)	(9,4)	0	1	swap

The resulting wall is

$$W' = (3, 8, 4, 7, 2, 9).$$

It has exactly the same frequency vector as W, but reading the three pairs now yields `101`.

5. Decode

The receiver needs only the final wall W' and password K:

- Compute $S(W')$. Frequency preservation gives $S(W') = S(W)$.
- Recreate $\pi_{K,S}$ and its consecutive pairs.
- Skip equal-value pairs and read the remaining pair orders as bits.
- Read the first 32 bits as a big-endian payload length, then read that many bytes.

With the correct password, the original bytes are recovered. A wrong password usually produces a different pairing and therefore unrelated bits. The browser decoder displays a valid UTF-8 result only when the declared length fits and the decoded bytes are valid UTF-8; otherwise it shows a deterministic lossy rendering of the extracted bytes.

Capacity and sizing

A random pair is unusable only when both independent samples select the same one of 30 variants:

$$\Pr[\text{equal}] = \frac{1}{30}, \qquad \Pr[\text{usable}] = \frac{29}{30}.$$

There are approximately $N/2$ pairs, so expected capacity is

$$\mathbb{E}[C] \approx \frac{N}{2} \cdot \frac{29}{30} = \frac{29N}{60} \text{ bits.}$$

A message of m bytes needs $32 + 8m$ usable pairs/bits. The generator starts from this estimate with a safety margin, then measures actual capacity and grows the wall until it fits.

Security model and limitations

- **The password is the only secret.** The wall, token counts, salt, and algorithm are public.
- **The salt does not add password entropy.** It only ensures a pairing is specific to the wall's multiset of tokens.
- **There is no authenticity or tamper detection.** Reordering, adding, or removing words can corrupt decoding.
- **This is not a password verifier.** A wrong password can occasionally yield a plausible length and valid UTF-8 by chance; do not treat the UI result as a cryptographic proof of correctness.
- **Wall size is a weak security lever.** Rebuilding the pairing costs $O(N)$ work per password guess, whereas password entropy grows exponentially. Use a strong passphrase and encrypt the payload separately when confidentiality matters.

Usage

Generate and read a wall in Python:

```
from fog.generator import generate_wall, read_wall

wall = generate_wall(b"secret message", key="THE_PASSPHRASE", n=6000)
text = " ".join(wall)

recovered = read_wall(text.split(), "THE_PASSPHRASE")
assert recovered == b"secret message"
```

Run the built-in round-trip check:

```
python3 fog/generator.py
```

To use the browser decoder, serve the directory rather than opening the page with `file://` (Web Crypto and ES modules require an HTTP origin):

```
cd fog
python3 -m http.server 8000
```

Then open `http://localhost:8000/decoder.html`.
